

Firebase AI Offline and On Device Capability Verification Plan

Implementation and test strategy for the AI Native Mastery Platform

Document	Implementation and capability verification plan
Version	0.1
Date	18 September 2026
Primary target	Android personal-device prototype
Decision required	Whether Firebase AI on-device inference can be an optional enhancement for the pilot

1 Recommendation

Use a hybrid test strategy rather than pure test-driven development for the entire effort. Begin with a time-boxed capability spike on real Android devices because Firebase AI hybrid inference is experimental and hardware-dependent. As soon as a behavior is confirmed, express it as an automated contract, integration, or acceptance test. Use conventional TDD from the start for the application-owned offline contract, deterministic fallbacks, event persistence, synchronization, mastery rules, and response validation.

Expected architectural conclusion. Firebase AI on-device generation should remain an optional enhancement, not a dependency of the rural-access MVP. The current Android path is available only on a limited set of relatively capable devices. The application must deliver the complete Diagnose, Learn, Practice, Verify, and Revisit loop on unsupported low-cost devices through authored content and deterministic logic.

2 Objectives

- Verify that supported Android devices can run useful short tutor prompts with all network access disabled after model preparation.
- Verify the exact routing behavior of ONLY_ON_DEVICE, PREFER_ON_DEVICE, PREFER_IN_CLOUD, and ONLY_IN_CLOUD modes.
- Measure availability, download size and time, warm-up latency, generation latency, memory pressure, battery impact, quota behavior, and output quality.
- Prove that unsupported devices receive an explicit authored fallback and never enter a broken or misleading AI state.
- Determine which GenAI use cases are safe for on-device execution and which must remain cloud-only or deferred.
- Produce a go, conditional-go, or no-go decision supported by reproducible evidence.

3 Current Platform Constraints

Constraint	Current documented behavior	Design consequence
Release status	Firebase AI hybrid inference on Android is experimental; the ML Kit Prompt API is beta.	Pin SDK versions, isolate the integration, and expect breaking changes.
Device coverage	On-device Prompt API support is limited to listed devices and Gemini Nano versions.	Test at least one supported device and multiple representative unsupported low-cost devices.

Constraint	Current documented behavior	Design consequence
Interaction model	On-device supports single-turn text generation and one Bitmap image; multi-turn chat is unavailable.	Implement tutoring as bounded turns with application-managed context, not an assumed chat session.
Output format	Structured output is not available on the Android on-device path.	Do not require JSON from local inference; validate simple text or use authored fallbacks.
Context limit	On-device requests are limited to 4,000 tokens and short outputs are recommended.	Use compact source excerpts and target outputs below 256 tokens.
Execution	Inference is foreground-only and subject to per-app burst and battery quotas.	Handle BUSY, battery quota, and background-blocked errors without losing learner work.
Language	English and Korean are documented as validated for on-device inference.	English-only pilot remains aligned; future languages require separate validation.
Observability	Firebase AI monitoring does not include on-device inference data.	Add privacy-preserving local telemetry and synchronize aggregate results later.

4 Architecture Under Test

Place Firebase AI behind a product-owned GenAI gateway. The learning loop calls a capability-level operation such as `explainMisconception` or `generateFollowUp` rather than calling a model SDK directly. The gateway chooses an authored response, on-device inference, or cloud inference according to an explicit policy and returns provenance with every result.

Component	Responsibility
Capability detector	Report SDK availability, supported device status, model status, connectivity, app foreground state, and policy eligibility.
Inference router	Apply an explicit mode and prohibit unexpected cloud fallback in verified offline sessions.
Prompt builder	Assemble compact approved context, redact unnecessary personal data, and enforce input and output budgets.
Response validator	Check non-empty output, length, prohibited answer leakage, grounding references where applicable, and safe failure behavior.
Fallback provider	Return authored explanations, hints, and questions for every required learner state.
Provenance recorder	Store provider, model or base-model name, SDK version, inference location, prompt policy, timing, error, and fallback reason.

5 Development Approach

5.1 Why pure TDD is not sufficient

A unit test cannot prove that a particular field device exposes Gemini Nano, that AICore finishes a model download, or that the SDK routes an actual request as documented. These are empirical capability questions. Writing production abstractions around unverified assumptions would create false confidence.

5.2 Where TDD should be mandatory

Area	Test-first expectation
Offline contract	A failing acceptance test proves the full learner loop cannot contact the network and still completes with authored fallbacks.
Routing policy	Unit and integration tests cover every capability, connectivity, inference-mode, and error combination before implementation.
Local persistence	Tests prove attempts and AI events are committed before any request and survive termination, restart, and reconnect.
Synchronization	Contract tests prove idempotency, ordering, conflict handling, and preservation of original inference provenance.
Mastery authority	Tests prove generative output cannot directly mark a skill Demonstrated or bypass rubric and evidence rules.
Response handling	Property and fixture tests cover empty, long, malformed, unsafe, irrelevant, and answer-leaking responses.

5.3 Recommended red green learn cycle

For external capabilities, use a short cycle: state a falsifiable hypothesis, run the smallest real-device experiment, record the result, then add a regression or contract test before integrating the behavior. This retains the discipline of TDD without pretending the third-party platform is already understood.

6 Test Environments and Device Matrix

Tier	Minimum coverage	Purpose
Unsupported low-cost	Two representative Android devices near the pilot hardware floor	Prove that the core loop and authored fallback remain complete and understandable.
Supported device	At least one device explicitly listed for the ML Kit Prompt API	Verify model discovery, download, warm-up, offline generation, inference location, and quotas.
Supported alternate Nano version	One device with a different documented Gemini Nano base-model version when available	Detect prompt-quality and latency differences across device-managed models.
Emulator or CI	Android emulator plus fake GenAI gateway	Run deterministic routing, persistence, UI, sync, and failure tests on every change.
Network-controlled lab	Wi-Fi proxy or firewall plus airplane-mode runs	Prove whether bytes leave the device and distinguish offline behavior from silent cloud fallback.

7 Capability Verification Backlog

ID	Test	Pass condition
CAP-01	SDK and device status detection	The app distinguishes unavailable, downloadable, downloading, available, and error states without blocking learning.
CAP-02	Model download and interruption	Download resumes or fails visibly; no partial state is presented as ready.

ID	Test	Pass condition
CAP-03	Warm-up behavior	Warm-up improves or characterizes first-response latency without freezing the UI.
OFF-01	ONLY_ON_DEVICE in airplane mode	A useful response is produced on a supported prepared device and network capture shows zero external requests.
OFF-02	Unsupported device in airplane mode	The app immediately uses the authored fallback and records the reason.
ROUTE-01	All four inference modes	Observed inference location and errors match policy for online, offline, supported, and unsupported combinations.
ROUTE-02	No hidden cloud fallback	Verified offline policy cannot produce an in-cloud result; any attempted violation fails closed to authored content.
LIMIT-01	Context and output limits	Oversized context is compacted or rejected before SDK invocation; output stays within the product budget.
LIFE-01	Background and lifecycle interruption	Background-blocked, rotation, process death, and restart do not lose the learner attempt or show a false completion.
QUOTA-01	Burst and battery quota	BUSY and battery quota errors trigger bounded backoff or authored fallback, never an infinite retry.
QUAL-01	Tutor quality fixture set	Human reviewers judge groundedness, usefulness, non-disclosure of answers, and reading level against predefined thresholds.
SYNC-01	Deferred telemetry and evidence sync	Local events synchronize idempotently with their original inference location and fallback reason intact.

8 Prompt and Quality Evaluation

Use a fixed evaluation set drawn from the initial Data Problem-Solving and Automation pathway. Include common misconceptions in tables, missing values, unit normalization, spreadsheet formulas, SQL joins, aggregation, and validation. Each case contains an approved explanation, prohibited answer content, learner attempt, expected misconception, and reviewer rubric.

Criterion	Measurement
Groundedness	Claims remain within the supplied skill and approved source excerpts.
Targeting	The response addresses the demonstrated misconception rather than restating the lesson.
Pedagogical usefulness	The response explains or probes without simply giving the protected answer.
Consistency	Repeated runs remain acceptably aligned across supported Nano versions and cloud fallback.
Accessibility	Response is concise, readable, and usable on a small screen; authored text remains available.

Criterion	Measurement
Safety and privacy	Prompts contain only required learner context and outputs avoid sensitive inference.

9 Phased Implementation

Phase	Duration	Work	Exit condition
0 Contract baseline	2-3 days	Define gateway, inference provenance, fake provider, offline acceptance test, and authored fallback fixtures.	Core loop passes with no Firebase AI dependency.
1 Capability spike	3-5 days	Build a minimal Kotlin probe for status, download, warm-up, inference modes, inference location, and airplane-mode execution.	Real-device evidence answers the primary feasibility questions.
2 Test harness	4-6 days	Convert findings into routing, lifecycle, quota, network, persistence, and sync tests.	Known behaviors and failures are repeatable.
3 Tutor vertical slice	5-7 days	Integrate one misconception explanation and one follow-up question with authored fallback.	A learner can complete the slice on supported and unsupported devices.
4 Device and quality evaluation	5-7 days	Run device matrix, latency and energy measurements, network capture, and human prompt-quality review.	Decision gates have sufficient evidence.
5 Decision and hardening	2-3 days	Document go or no-go, pin versions, finalize feature flags, and prepare demo evidence.	Prototype scope and fallback policy are approved.

10 Release Gates

- The complete learning loop passes in airplane mode on every target device, including devices without on-device GenAI.
- ONLY_ON_DEVICE produces no observed network traffic and never silently reports an in-cloud response.
- Unsupported or unavailable states resolve to authored content without a crash, indefinite spinner, or lost learner attempt.
- Every AI result records its inference location, model or base-model identifier when available, SDK version, prompt policy, and fallback status.
- No generative response can directly update mastery or bypass versioned validators and rubrics.
- The selected on-device tutor prompts meet the agreed human quality threshold across the tested supported devices.
- Latency, memory, battery, quota, and model-download behavior are acceptable for the pilot workflow and documented for users.

11 Decision Outcomes

Outcome	Meaning	Product action
Go	Supported-device coverage and quality are adequate for the pilot cohort.	Offer on-device GenAI behind capability detection while keeping authored fallbacks.

Outcome	Meaning	Product action
Conditional go	The capability works but coverage, quality, or operational limits exclude much of the cohort.	Use on-device GenAI only as an enhancement or demonstration path; measure uptake separately.
No-go for pilot	Device coverage, reliability, quality, or maintenance risk is incompatible with the access goal.	Retain Firebase for cloud inference and synchronization; use authored offline tutoring on learner devices.

12 Deliverables

- Minimal Android capability probe and reproducible setup notes.
- GenAI gateway interface, fake provider, authored fallback provider, and provenance schema.
- Automated unit, contract, integration, lifecycle, and offline acceptance test suite.
- Real-device compatibility and performance matrix.
- Prompt-quality evaluation set and reviewer results.
- Network-capture evidence for verified offline runs.
- Go, conditional-go, or no-go recommendation with prototype scope changes.

13 References

- Firebase AI Logic hybrid overview: <https://firebase.google.com/docs/ai-logic/hybrid>
- Firebase AI Logic Android hybrid guide: <https://firebase.google.com/docs/ai-logic/hybrid/android/get-started>
- ML Kit GenAI overview and supported devices: <https://developers.google.com/ml-kit/genai>
- Firebase AI Logic solutions and production controls: <https://firebase.google.com/docs/ai-logic/solutions/overview>
- Firebase AI Logic pricing: <https://firebase.google.com/docs/ai-logic/pricing>